PRODUCTION GUIDE LANGGRAPH STATEGRAPH TAVILY WEB SEARCH

LangGraph, Tavily Search & Gemini AI Integration Guide

Node.js Express me LangGraph StateGraph, Tavily Real-Time Web Search, Internal MongoDB Ledger Tools, Multi-Turn MemorySaver & Google Gemini 2.5 Flash

> **Hybrid AI Agent (Internal Ledger + Live Web Search):**
> Is architecture me agent ke paas **Do Duniya Ka Access** hota hai: 1. **Internal Private Data:** User ka bank balance aur transactions (MongoDB Double-Entry Ledger). 2. **External Public Web:** Live financial news, exchange rates (USD to INR), RBI guidelines, ya koi bhi internet question via **Tavily Search**!

Part 1: API Keys Setup (Gemini + Tavily)

1. 100% Free Gemini API Key (Without Credit Card):

1. Browser me **https://aistudio.google.com/api-keys** open karo.
2. Click **"Create API key"** ➔ Select karo **"Create API key in new project"** (Purana project select mat karna warna 403 error aayega).
3. Key copy karo aur `.env` me paste karo.

2. Free Tavily Search API Key (1,000 Free Searches / Month):

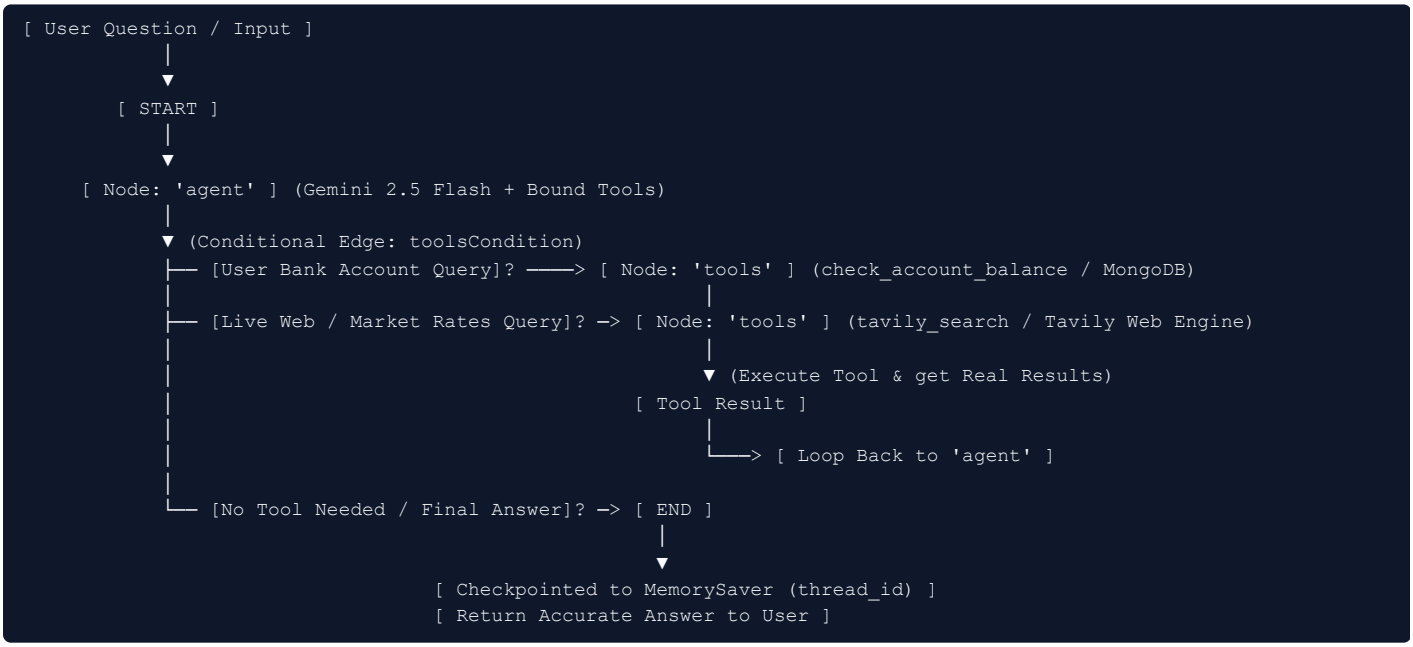
1. Browser me **https://app.tavily.com/** open karo (Google / GitHub se 1-click free sign-up).
2. Dashboard par tumhari free API key dikhegi (format: `tvly-...`).
3. Copy karke `.env` file me paste karo:

```
# .env File
GEMINI_API_KEY=AIzaSyD-YourCopiedKeyHere1234567890
TAVILY_API_KEY=tvly-YourTavilyKeyHere1234567890
```

Part 2: Packages Installation

```
npm install @langchain/langgraph @langchain/google-genai @langchain/tavily @langchain/core zod dotenv
```

Part 3: Complete StateGraph Architecture with Tavily Search



Part 4: Production LangGraph Service (src/services/langgraph.agent.service.js)

```
// src/services/langgraph.agent.service.js
const { StateGraph, MessagesAnnotation, START, END, MemorySaver } = require("@langchain/langgraph");
const { ToolNode, toolsCondition } = require("@langchain/langgraph/prebuilt");
const { ChatGoogleGenerativeAI } = require("@langchain/google-genai");
const { TavilySearch } = require("@langchain/tavily");
const { tool } = require("@langchain/core/tools");
const { z } = require("zod");
const { SystemMessage, HumanMessage } = require("@langchain/core/messages");
const accountModel = require("../models/account.model");
const transactionModel = require("../models/transaction.model");

// In-Memory Checkpointer for Stateful Multi-Turn Conversations
const memoryCheckpointer = new MemorySaver();
```

```
// 1. Initialize Gemini Model
const getGeminiModel = () => {
    return new ChatGoogleGenerativeAI({
        model: "gemini-2.5-flash",
        apiKey: process.env.GEMINI_API_KEY,
        temperature: 0.2,
        maxRetries: 2
    });
};

// 2. Tavily Web Search Tool Initializer
const getTavilySearchTool = () => {
    const rawKey = process.env.TAVILY_API_KEY ? process.env.TAVILY_API_KEY.trim() : "";
    if (rawKey && rawKey.startsWith("tvly-")) {
        return new TavilySearch({ maxResults: 3, topic: "general" });
    }
    // Graceful fallback if key not yet entered
    return tool(
        async ({ query }) => JSON.stringify({
            notice: "TAVILY_API_KEY not configured in .env",
            guidance: "Get free key from https://app.tavily.com/ and add TAVILY_API_KEY=tvly-... in .env",
            query
        }),
        { name: "tavily_search", description: "Search live web for current news, market rates, and facts.", schema:
z.object({ query: z.string() }) }
    );
};

// 3. User-Scoped Banking Tools + Tavily Search
const createBankingTools = (userId) => {
    // Tool 1: Live Ledger Balance
    const checkBalance = tool(
        async () => {
            const accounts = await accountModel.find({ user: userId });
            const list = [];
            let total = 0;
            for (const acc of accounts) {
                const bal = await acc.getBalance();
                total += bal;
                list.push({ accountId: acc._id, balance: bal, currency: acc.currency });
            }
            return JSON.stringify({ accounts: list, totalBalance: total });
        },
        { name: "check_account_balance", description: "Fetch live balance from immutable ledger.", schema: z.object
({}) }
    );

    // Tool 2: Recent Transactions
    const getTransactions = tool(
        async ({ limit }) => {
            const count = Math.min(limit || 5, 20);
            const accounts = await accountModel.find({ user: userId });
            const accountIds = accounts.map(a => a._id);
            const rawTx = await transactionModel.find({
                $or: [{ fromAccount: { $in: accountIds } }, { toAccount: { $in: accountIds } }]
            }).sort({ createdAt: -1 }).limit(count).lean();

            return JSON.stringify(rawTx.map(tx => ({
                type: accountIds.some(id => id.toString() === tx.fromAccount?.toString()) ? "DEBIT" : "CREDIT",
                amount: tx.amount, date: tx.createdAt
            })));
        },
        { name: "get_recent_transactions", description: "Fetch recent debit and credit transaction history of use
r.", schema: z.object({ limit: z.number().optional().default(5) }) }
    );

    // Tool 3: Tavily Search
    const webSearch = getTavilySearchTool();

    return [checkBalance, getTransactions, webSearch];
};

// 4. Build & Compile LangGraph StateGraph Workflow
const buildBankingStateGraph = (userId) => {
    const model = getGeminiModel();
    const tools = createBankingTools(userId);
    const modelWithTools = model.bindTools(tools);
```

```
    const systemPrompt = new SystemMessage(`
You are VaultX Banking AI Agent powered by LangGraph & Tavily.
Routing Rules:
- For user's bank accounts: ALWAYS invoke check_account_balance or get_recent_transactions.
- For current world events, real-time rates, financial news, or web facts: ALWAYS invoke tavily_search.
- NEVER invent facts or numbers. Answer in friendly Hinglish/English.
`);

    const agentNode = async (state) => {
        const messages = state.messages.some(m => m instanceof SystemMessage)
            ? state.messages
            : [systemPrompt, ...state.messages];
        const res = await modelWithTools.invoke(messages);
        return { messages: [res] };
    };

    const toolNode = new ToolNode(tools);

    const workflow = new StateGraph(MessagesAnnotation)
        .addNode("agent", agentNode)
        .addNode("tools", toolNode)
        .addEdge(START, "agent")
        .addConditionalEdges("agent", toolsCondition) // If tool_calls -> 'tools', else -> END
        .addEdge("tools", "agent");                   // Loop result back to agent

    return workflow.compile({ checkpointer: memoryCheckpointer });
};

// 5. Run with Stateful Thread ID
const runLangGraphBankingAgent = async (userId, userMessage, threadId = null) => {
    const graph = buildBankingStateGraph(userId);
    const activeThreadId = threadId || `user_thread_${userId}`;

    const finalState = await graph.invoke(
        { messages: [new HumanMessage(userMessage)] },
        { configurable: { thread_id: activeThreadId } }
    );

    const messages = finalState.messages;
    const lastMessage = messages[messages.length - 1];

    return {
        reply: lastMessage.content,
        threadId: activeThreadId,
        totalMessagesInThread: messages.length,
        toolsUsed: messages.some(m => m._getType?.() === "tool" || m.constructor?.name === "ToolMessage")
    };
};

module.exports = { runLangGraphBankingAgent, buildBankingStateGraph };
```

Part 5: Live Testing (Banking Queries vs Web Search Queries)

Test 1: User Asks Personal Account Query ➔ Calls `check_account_balance` (MongoDB)

```
curl -X POST http://localhost:3000/api/ai/graph \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"message": "Mera account balance kitna hai?"}'
```

```
{
  "success": true,
  "framework": "langgraph",
  "toolsUsed": true,
  "reply": "Aapke VaultX Savings Account me kul ₹45,000 ka balance uplabdh hai."
}
```

Test 2: User Asks Live Market / Web Query ➔ Calls `tavily_search` (Tavily Engine)

```
curl -X POST http://localhost:3000/api/ai/graph \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"message": "Current USD to INR exchange rate kya chal raha hai?"}'
```

```
{
  "success": true,
  "framework": "langgraph",
  "toolsUsed": true,
  "reply": "Tavily live search ke mutabiq, current USD to INR exchange rate lagbhag ₹84.40 chal raha hai."
}
```

Part 6: Top 5 Production Errors & Instant Fixes

Error / Symptom	Kyun Aata Hai?	Instant 1-Minute Solution
`Tavily API key not found`	`TAVILY_API_KEY` .env me set nahi hai ya empty hai.	**Fix:** app.tavily.com se free key le kar `.env` me `TAVILY_API_KEY=tvly-...` paste karo. Humara code bina crash huye graceful fallback deta hai.
`403 PERMISSION_DENIED (free tier not available)`	Gemini key purane Google Cloud project me bani thi.	**Fix:** AI Studio me select karo **"Create API key in new project"**.
`Infinite Graph Recursion Error`	Agent node aur Tool node ke beech me exit condition nahi mili.	**Fix:** `toolsCondition` prebuilt edge use karo jo non-tool calls par `END` par bhejta hai.
`TypeError: Cannot read properties (reading 'configurable')`	Compile karte waqt checkpointer pass nahi kiya.	**Fix:** `workflow.compile({ checkpointer: new MemorySaver() })` pass karo.
`429 Resource Exhausted`	Free rate-limit cross ho gayi.	**Fix:** Redis Caching layer aur sliding window rate limiter use karo.

Interview Ready Pitch:
*"Maine VaultX banking backend me **LangGraph StateGraph** ke sath **Tavily Search API** integrate kiya hai using `@langchain/tavily`. Isse humara agent hybrid ban chuka hai — personal account queries par wo authenticated MongoDB double-entry ledger tools invoke karta hai, aur live financial market trends ya forex rates par wo Tavily real-time web search tool invoke karta hai. Graph me conditional edge routing (`toolsCondition`) aur `MemorySaver` checkpointer ke sath stateful multi-turn conversation memory maintain ki gayi hai."*